This is a great collection of SQL scripts! You've set up a complete bookstore database schema and a comprehensive set of practice queries, moving from basic retrieval to more advanced analysis.

Here is your code organized into a clean "booklet" format for easy reference.

---

# 1. Database Schema

Here are the `CREATE TABLE` statements for your three main tables.

### Customers Table

SQL
```
CREATE TABLE Customers(
        Customer_ID INT PRIMARY KEY,
        Name VARCHAR(50) NOT NULL,
        Email VARCHAR(100) NOT NULL,
        Phone INT,
        City VARCHAR(100),
        Country VARCHAR(100)
);
```

### Books Table

SQL
```
CREATE TABLE Books(
        Book_ID INT PRIMARY KEY,
        Title VARCHAR(100) NOT NULL,
        Author VARCHAR(100) NOT NULL,
        Genre VARCHAR(100) NOT NULL,
        Published_Year INT,
        Price NUMERIC(10,2),
        Stock INT
);
```

### Orders Table

SQL
```
CREATE TABLE Orders(
        Order_ID INT PRIMARY KEY,
        Customer_ID INT REFERENCES Customers(Customer_ID),
        Book_ID INT REFERENCES Books(Book_ID),
        Order_Date DATE,
        Quantity INT,
        Total_Amount NUMERIC(10,2)
);
```

---

# 2. Data Loading

This is the example command for loading data into the `Customers` table from a CSV file.

SQL
```
COPY
Customers(Customer_ID,Name,Email,Phone,City,Country)
FROM '/Users/nirajsahani/Self-Coding/8.SQL/employee_data.csv'
DELIMITER','
CSV HEADER;
```

---

# 3. Basic Queries

A set of fundamental queries for data retrieval.

**1. Retrieve all books in the "Fiction" genre:**

SQL
```
SELECT title, author, published_year, price, stock, genre
FROM Books
WHERE genre='Fiction';
```

**2. Find books published after the year 1950:**

SQL
```
SELECT title, author, published_year, price, stock, genre
FROM Books
WHERE published_year >= 1950;
```

**3. List all customers from the Canada:**

SQL
```
SELECT * FROM Customers
WHERE country='Canada';
```

**4. Show orders placed in October 2023:** *(Note: Your comment mentioned November, but your queries correctly filter for October '2023-10-1' to '2023-10-30'.)*

SQL
```
-- Solution A: Using AND
SELECT * FROM Orders
WHERE order_date >= '2023-10-1' AND order_date <= '2023-10-30';

-- Solution B: Using BETWEEN
SELECT * FROM Orders
WHERE order_date BETWEEN '2023-10-1' AND '2023-10-30';
```

**5. Retrieve the total stock of books available:**

SQL
```
SELECT SUM(stock) as Total_aiviable_book_in_stock
FROM Books;
```

**6. Find the details of the most expensive book:**

SQL

```
-- Solution A: Using ORDER BY and LIMIT
SELECT * FROM Books
ORDER BY price DESC
LIMIT 1;

-- Solution B: Using DENSE_RANK (handles ties)
SELECT *
FROM (
  SELECT
    *,
    DENSE_RANK() OVER (ORDER BY price DESC) AS price_rank
  FROM Books
) ranked
WHERE price_rank = 1;
```

**7. Show all customers who ordered more than 1 quantity of a book:**

SQL

```
SELECT C.*, O.quantity
FROM Customers C
JOIN Orders O
ON C.customer_id = O.customer_id
WHERE O.quantity >= 1;
```

**8. Retrieve all orders where the total amount exceeds $20:**

SQL

```
SELECT * FROM Orders
WHERE total_amount > 20;
```

**9. List all genres available in the Books table:**

SQL

```
SELECT DISTINCT genre
FROM Books;
```

**10. Find the book with the lowest stock:**

SQL

```
-- Solution A: Using ORDER BY and LIMIT
SELECT * FROM Books
ORDER BY stock
LIMIT 1;

-- Solution B: Using DENSE_RANK (handles ties)
SELECT * FROM(
               SELECT
               *,
                DENSE_RANK() OVER (ORDER BY stock) as stock_rank
               FROM Books
       ) ranked
WHERE stock_rank=1;
```

**11. Calculate the total revenue generated from all orders:**

SQL

```
SELECT SUM(total_amount) AS total_revenue
```

```
FROM Orders;
```

---

# 4. Advanced Queries

More complex queries involving joins, aggregations, and subqueries.

### 1. Retrieve the total number of books sold for each genre:

SQL
```
SELECT B.genre, SUM(O.quantity) as total_sum_per_by_genre
FROM Books B
JOIN Orders O
ON B.book_id = O.book_id
GROUP BY (B.genre);
```

### 2. Find the average price of books in the "Fantasy" genre:

SQL
```
SELECT genre, AVG(price) AS average_price
FROM Books
WHERE genre='Fantasy'
Group by(genre);
```

### 3. List customers who have placed at least 2 orders:

SQL
```
-- Solution A: Querying Orders table only
SELECT customer_id, COUNT (Order_id) AS ORDER_COUNT
FROM orders
GROUP BY customer_id
HAVING COUNT (Order_id) >= 2;

-- Solution B: Joining with Customers to get details
SELECT C.*, Count(O.customer_id) as total_count
FROM Customers C
JOIN Orders O
ON C.customer_id = O.customer_id
GROUP BY (C.customer_id)
HAVING Count(O.customer_id) >= 2;
```

### 4. Find the most frequently ordered book (by number of orders):

SQL
```
-- Solution A: Simple LIMIT
SELECT book_id, count(order_id) as frequency
FROM Orders
GROUP BY(Book_id)
order by frequency desc
limit 1;

-- Solution B: Using DENSE_RANK (handles ties for frequency)
SELECT *
FROM (
  SELECT
    B.title,
```

```
    COUNT(O.book_id) AS frequency,
    SUM(O.quantity) AS TOTAL_QUANTITY_SUM,
    DENSE_RANK() OVER (ORDER BY COUNT(O.book_id) DESC) AS rank
  FROM Books B
  JOIN Orders O
    ON B.book_id = O.book_id
  GROUP BY B.title
) AS ranked_books
WHERE rank = 1;

-- Solution C: Using CTE (handles ties for frequency and picks highest
quantity)
WITH ranked_books AS (
  SELECT
    B.title,
    COUNT(O.book_id) AS frequency,
    SUM(O.quantity) AS TOTAL_QUANTITY_SUM,
    DENSE_RANK() OVER (ORDER BY COUNT(O.book_id) DESC) AS rank
  FROM Books B
  JOIN Orders O
    ON B.book_id = O.book_id
  GROUP BY B.title
)
SELECT *
FROM ranked_books
where rank = 1 AND TOTAL_QUANTITY_SUM = (SELECT max(TOTAL_QUANTITY_SUM)
from ranked_books where rank=1);
```

### 5. Show the top 3 most expensive books of 'Fantasy' Genre:

SQL

```
SELECT title, genre, book_id, price
from Books
where genre='Fantasy'
order by price desc
limit 3;
```

### 6. Retrieve the total quantity of books sold by each author:

SQL

```
SELECT B.author, SUM(O.quantity) AS total_quantity
FROM Books B
JOIN Orders O
ON B.book_id = O.book_id
GROUP BY (B.author);
```

### 7. List the cities where customers who spent over $30 are located:

SQL

```
-- Interpretation 1: Cities where ANY single order was > $30
SELECT DISTINCT c.city, o.total_amount
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
WHERE o.total_amount > 30;

-- Interpretation 2: Cities where the SUM of all orders from that city is >
$30
SELECT C.city, SUM(O.total_amount) as city_wise_spend
```

```
FROM Customers C
JOIN Orders O
ON C.customer_id = O.customer_id
GROUP BY (C.city)
HAVING SUM(O.total_amount) > 30
ORDER BY city_wise_spend DESC;

-- Interpretation 3: Customers (and their city) who spent > $30 in total
SELECT C.name, C.city, SUM(O.total_amount) as city_wise_spend
FROM Customers C
JOIN Orders O
ON C.customer_id = O.customer_id
GROUP BY (C.name, C.city)
HAVING SUM(O.total_amount) > 30
ORDER BY city_wise_spend DESC;
```

### 8. Find the customer who spent the most on orders:

SQL

```
-- Interpretation 1: Customer who spent the most in TOTAL (sum of all their
orders)
SELECT C.name, SUM(O.total_amount) as Total_spend_Customer_wise
FROM Customers C
JOIN Orders O
ON C.customer_id = O.customer_id
GROUP BY (C.name)
ORDER BY Total_spend_Customer_wise DESC;

-- Interpretation 2: Customer associated with the single largest order
SELECT C.name, O.total_amount
FROM Customers C
JOIN Orders O
ON C.customer_id = O.customer_id
order by O.total_amount DESC;
```

### 9. Calculate the stock remaining:

SQL

```
-- Interpretation 1: Remaining stock PER BOOK
SELECT
  B.book_id,
  B.title,
  B.stock AS Total_available,
  COALESCE(SUM(O.quantity), 0) AS Total_Sold_unit,
  B.stock - COALESCE(SUM(O.quantity), 0) AS Remaining_stock
FROM Books B
LEFT JOIN Orders O
  ON B.book_id = O.book_id
GROUP BY B.book_id, B.title
ORDER BY B.book_id;

-- Interpretation 2: Total remaining stock of ALL books
WITH data1 AS(
        SELECT B.book_id,
                SUM(B.stock) as total_stock,
                COALESCE (SUM (O.quantity),0) AS Order_quantity
        FROM Books B
        LEFT JOIN Orders O
        ON B.book_id=O.book_id
```

```
        GROUP BY b.book_id
)
SELECT
Sum(total_stock) - sum(Order_quantity) as total_reaming from data1;
```

---

# 5. Final Data Checks

Simple `SELECT *` queries to view all data in the tables.

SQL

```
SELECT * FROM Customers;
SELECT * FROM Books;
SELECT * FROM Orders;
```